

1. Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.

Program:

cart.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Cart</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1 class="page-title">Your Cart</h1>
<div class="cart-container">
  <div class="cart-item">
    <span>Product 1</span>
    <span>$10.00</span>
  </div>
  <div class="cart-item">
    <span>Product 3</span>
    <span>$20.00</span>
  </div>
  <div class="cart-item">
    <span>Product 4</span>
    <span>$8.00</span>
  </div>
</div>
<a href="catalog.html" class="cart-link">Back to Catalog</a>
</body>
</html>
```

catalog.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Catalog</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1 class="page-title">Product Catalog</h1>
<div class="products-grid">
```

```
<div class="product-card">
  <h3>Product 1</h3>
  <p>Description: High-quality item for daily use.</p>
  <p>Price: $10.00</p>
  <button>Add to Cart</button>
</div>
<div class="product-card">
  <h3>Product 2</h3>
  <p>Description: Durable and affordable.</p>
  <p>Price: $15.00</p>
  <button>Add to Cart</button>
</div>
</div>
<a href="cart.html" class="cart-link">Go to Cart</a>
</body>
</html>
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Catalog</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 class="page-title">Product Catalog</h1>
  <div class="products-grid">
    <div class="product-card">
      <h3>Product 1</h3>
      <p>Description: High-quality item for daily use.</p>
      <p>Price: $10.00</p>
      <button>Add to Cart</button>
    </div>
    <div class="product-card">
      <h3>Product 2</h3>
      <p>Description: Durable and affordable.</p>
      <p>Price: $15.00</p>
      <button>Add to Cart</button>
    </div>
  </div>
  <a href="cart.html" class="cart-link">Go to Cart</a>
</body>
</html>
```

register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Catalog</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1 class="page-title">Product Catalog</h1>
<div class="products-grid">
  <div class="product-card">
    <h3>Product 1</h3>
    <p>Description: High-quality item for daily use.</p>
    <p>Price: $10.00</p>
    <button>Add to Cart</button>
  </div>
  <div class="product-card">
    <h3>Product 2</h3>
    <p>Description: Durable and affordable.</p>
    <p>Price: $15.00</p>
    <button>Add to Cart</button>
  </div>
</div>
<a href="cart.html" class="cart-link">Go to Cart</a>
</body>
</html>
```

style.css

```
/* ----- Global Styles ----- */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background: #fff;
  color: #000;
}
h1, h2, h3 {
  margin: 0 0 10px 0;
}
a {
  color: #000;
  text-decoration: none;
```

```
}
```

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 100vh;  
}
```

```
.form-box {  
  background: #f2f2f2;  
  padding: 20px;  
  border-radius: 8px;  
  width: 90%;  
  max-width: 400px;  
  display: flex;  
  flex-direction: column;  
}
```

```
.form-box input {  
  margin: 8px 0;  
  padding: 10px;  
  border-radius: 4px;  
  border: 1px solid #000;  
}
```

```
.form-box button {  
  padding: 10px;  
  background: #000;  
  color: #fff;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
  margin-top: 10px;  
}
```

```
.form-box p {  
  text-align: center;  
  margin-top: 10px;  
}
```

```
.page-title {  
  text-align: center;  
  margin: 20px 0;  
}
```

```
.products-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));  
  gap: 15px;
```

```
padding: 0 15px 30px 15px;
}
```

```
.product-card {
  background: #f2f2f2;
  padding: 15px;
  border-radius: 6px;
  display: flex;
  flex-direction: column;
}
```

```
.product-card h3 {
  margin-bottom: 5px;
}
```

```
.product-card p {
  margin-bottom: 8px;
}
```

```
.product-card button {
  padding: 8px;
  background: #000;
  color: #fff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  margin-top: auto;
}
```

```
.cart-container {
  max-width: 600px;
  margin: 20px auto;
  padding: 0 15px;
}
```

```
.cart-item {
  display: flex;
  justify-content: space-between;
  background: #f2f2f2;
  padding: 10px 15px;
  margin-bottom: 10px;
  border-radius: 6px;
}
```

```
.cart-link {
  display: inline-block;
  margin: 20px auto;
  padding: 10px 15px;
  background: #000;
```

```

color: #fff;
text-decoration: none;
border-radius: 4px;
text-align: center;
}
@media (max-width: 500px) {
.products-grid {
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
}
.cart-item {
  flex-direction: column;
  gap: 5px;
  align-items: flex-start;
}
}

```

Output:

Login

Username

Password

Login

Don't have an account? Register

Or go to Catalog directly

Create Account

Username

Email

Password

Register

Already have an account? Login

Or go to Catalog directly

Your Cart

Product 1	\$10.00
Product 3	\$20.00
Product 4	\$8.00

[Back to Catalog](#)

Product Catalog

Product 1

Description: High-quality item for daily use.

Price: \$10.00

[Add to Cart](#)

Product 2

Description: Durable and affordable.

Price: \$15.00

[Add to Cart](#)

[Go to Cart](#)

2. Make the above web application responsive web application using Bootstrap framework.

Program:

cart.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cart</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
  <div class="container mt-5">
    <h1 class="text-center mb-4">Your Cart</h1>
    <div class="list-group">
      <div class="list-group-item d-flex justify-content-between align-items-center">
        Product 1
        <span>$10.00</span>
      </div>
      <div class="list-group-item d-flex justify-content-between align-items-center">
        Product 3
        <span>$20.00</span>
      </div>
      <div class="list-group-item d-flex justify-content-between align-items-center">
        Product 4
        <span>$8.00</span>
      </div>
    </div>
    <div class="text-center mt-4">
      <a href="catalog.html" class="btn btn-dark">Back to Catalog</a>
    </div>
  </div>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

catalog.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Catalog</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
  <div class="container mt-5">
    <h1 class="text-center mb-4">Product Catalog</h1>
    <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
      <div class="col">
        <div class="card shadow-sm">
          <div class="card-body">
            <h5 class="card-title">Product 1</h5>
            <p class="card-text">Description: High-quality item for daily use.</p>
            <p class="card-text">Price: $10.00</p>
            <button class="btn btn-dark w-100">Add to Cart</button>
          </div>
        </div>
      </div>
      <div class="col">
        <div class="card shadow-sm">
          <div class="card-body">
            <h5 class="card-title">Product 2</h5>
            <p class="card-text">Description: Durable and affordable.</p>
            <p class="card-text">Price: $15.00</p>
            <button class="btn btn-dark w-100">Add to Cart</button>
          </div>
        </div>
      </div>
      <div class="col">
        <div class="card shadow-sm">
          <div class="card-body">
            <h5 class="card-title">Product 3</h5>
            <p class="card-text">Description: Best seller product with premium quality.</p>
            <p class="card-text">Price: $20.00</p>
            <button class="btn btn-dark w-100">Add to Cart</button>
          </div>
        </div>
      </div>
    </div>
    <div class="text-center mt-4">
      <a href="cart.html" class="btn btn-dark">Go to Cart</a>
    </div>
  </div>
</body>
</html>
```

```

</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
  <div class="container d-flex justify-content-center align-items-center min-vh-100">
    <div class="card p-4 w-100" style="max-width: 400px;">
      <h2 class="text-center mb-3">Login</h2>
      <form>
        <input type="text" class="form-control mb-3" placeholder="Username">
        <input type="password" class="form-control mb-3" placeholder="Password">
        <button class="btn btn-dark w-100 mb-3">Login</button>
      </form>
      <p class="text-center"><a href="register.html">Register</a> | <a
href="catalog.html">Catalog</a></p>
    </div>
  </div>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

```

register.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Register</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

```

```

<body class="bg-light">
  <div class="container d-flex justify-content-center align-items-center min-vh-100">
    <div class="card p-4 w-100" style="max-width: 400px;">
      <h2 class="text-center mb-3">Create Account</h2>
      <form>
        <input type="text" class="form-control mb-3" placeholder="Username">
        <input type="email" class="form-control mb-3" placeholder="Email">
        <input type="password" class="form-control mb-3" placeholder="Password">
        <button class="btn btn-dark w-100 mb-3">Register</button>
      </form>
      <p class="text-center"><a href="index.html">Login</a> | <a
href="catalog.html">Catalog</a></p>
    </div>
  </div>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

Output:

Login

Username

Password

Login

[Register](#) | [Catalog](#)

Create Account

Username

Email

Password

Register

[Login](#) | [Catalog](#)

Product Catalog

Product 1

Description: High-quality item for daily use.

Price: \$10.00

Add to Cart

Product 2

Description: Durable and affordable.

Price: \$15.00

Add to Cart

Product 3

Description: Best seller product with premium quality.

Price: \$20.00

Add to Cart

Go to Cart

Your Cart

Product 1	\$10.00
Product 3	\$20.00
Product 4	\$8.00

Back to Catalog

3. Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.

Program:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Shopping Cart App</title>
  <style>
    * { box-sizing: border-box; font-family: Arial, sans-serif; }
    body { margin: 0; background: #f4f4f4; }
    /* Layout */
    .container { max-width: 1100px; margin: auto; padding: 1rem; }
    header { background: #222; color: #fff; padding: 1rem; display: flex; justify-content:
space-between; }
    header button { background: #fff; border: none; padding: 0.4rem 0.7rem; cursor: pointer; }
    /* Forms */
    form { max-width: 400px; margin: 2rem auto; background: #fff; padding: 1.5rem; border-
radius: 5px; }
    input { width: 100%; padding: 0.6rem; margin: 0.5rem 0; }
    .error { color: red; font-size: 0.85rem; }
    /* Catalog */
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap:
1rem; }
    .card { background: #fff; padding: 1rem; border-radius: 5px; display: flex; flex-direction:
column; }
    .card button { margin-top: auto; }
    /* Cart */
    .cart { background: #fff; padding: 1rem; margin-top: 1rem; }
    .cart-item { display: flex; justify-content: space-between; }
    /* Utility */
    .hidden { display: none; }
    button { padding: 0.5rem; cursor: pointer; }
  </style>
</head>
<body>
<header>
  <div>My Shop</div>
  <button onclick="logout()">Logout</button>
</header>
```

```

<!-- Registration -->
<form id="registerForm">
  <h3>Register</h3>
  <input id="rUser" placeholder="Username" />
  <input id="rPass" type="password" placeholder="Password" />
  <div class="error" id="rErr"></div>
  <button type="submit">Register</button>
</form>
<!-- Login -->
<form id="loginForm" class="hidden">
  <h3>Login</h3>
  <input id="lUser" placeholder="Username" />
  <input id="lPass" type="password" placeholder="Password" />
  <div class="error" id="lErr"></div>
  <button type="submit">Login</button>
</form>
<!-- Catalog -->
<div id="shop" class="container hidden">
  <h2>Catalog</h2>
  <div class="grid" id="products"></div>
  <div class="cart">
    <h3>Cart</h3>
    <div id="cart"></div>
  </div>
</div>
<script>
const products = [
  { id: 1, name: "Phone", price: 500 },
  { id: 2, name: "Laptop", price: 1000 },
  { id: 3, name: "Headphones", price: 150 }
];
let cart = [];
// Registration
registerForm.onsubmit = e => {
  e.preventDefault();
  if (!rUser.value || !rPass.value) {
    rErr.textContent = "All fields required";
    return;
  }
  localStorage.setItem("user", rUser.value);
  localStorage.setItem("pass", rPass.value);
  registerForm.classList.add("hidden");
  loginForm.classList.remove("hidden");
};

```

```

// Login
loginForm.onSubmit = e => {
  e.preventDefault();
  if (!User.value !== localStorage.getItem("user") || !Pass.value !==
localStorage.getItem("pass")) {
    lErr.textContent = "Invalid credentials";
    return;
  }
  loginForm.classList.add("hidden");
  shop.classList.remove("hidden");
  loadProducts();
};

function loadProducts() {
  products.forEach(p => {
    productsDiv = document.createElement("div");
    productsDiv.className = "card";
    productsDiv.innerHTML = `<h4>${p.name}</h4><p>₹${p.price}</p><button
onClick="addToCart(${p.id})">Add</button>`;
    document.getElementById("products").appendChild(productsDiv);
  });
}

function addToCart(id) {
  cart.push(products.find(p => p.id === id));
  renderCart();
}

function renderCart() {
  cartDiv.innerHTML = "";
  cart.forEach(c => {
    cartDiv.innerHTML += `<div class='cart-
item'><span>${c.name}</span><span>₹${c.price}</span></div>`;
  });
}

function logout() {
  location.reload();
}
</script>
</body>
</html>

```

Output:

Register

All fields required

Login

Invalid credentials

My Shop

Catalog

Phone

₹500

Laptop

₹1000

Headphones

₹150

Cart

4. Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.

Program:

index.html

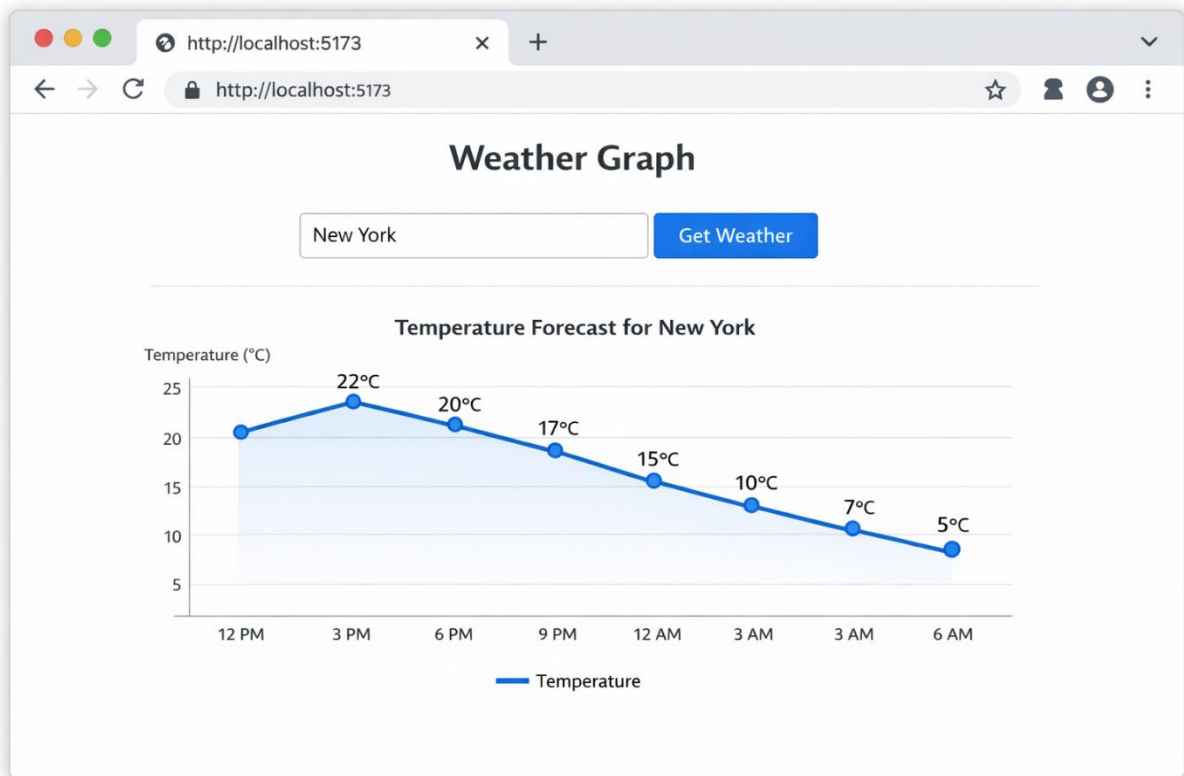
```
<!DOCTYPE html>
<html>
<head>
  <title>Weather App</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h2>Weather Information</h2>
<input type="text" id="city" placeholder="Enter City">
<button onclick="getWeather()">Get Weather</button>
<canvas id="weatherChart" width="400" height="200"></canvas>
<script src="script.js"></script>
</body>
</html>
```

script.js

```
const apiKey = "YOUR_API_KEY"; // get from openweathermap.org
let chart;
const getWeather = async () => {
  const city = document.getElementById("city").value;
  const response = await fetch(
    `https://api.openweathermap.org/data/2.5/forecast?q=${city}&units=metric&appid=${apiKey}`
  );
  const data = await response.json();
  const temps = data.list.slice(0, 5).map(item => item.main.temp);
  const times = data.list.slice(0, 5).map(item => item.dt_txt);
  drawChart(times, temps);
};
const drawChart = (labels, data) => {
  const ctx = document.getElementById("weatherChart").getContext("2d");
  if (chart) chart.destroy();
  chart = new Chart(ctx, {
    type: "line",
    data: {
      labels: labels,
      datasets: [{
        label: "Temperature (°C)",
        data: data,
```

```
borderWidth: 2    }]  }  });};
```

Output:



5. Develop a java standalone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.

Program:

DBConnection.java

```
package mypack;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DBConnection {
    private static final String URL =
"jdbc:oracle:thin:system/vnrvjiet@localhost:1521/XEPDB1";
    public static Connection getConnection() {
        try {
            System.out.println(" Connecting to Oracle 21c XE...");
            Connection con = DriverManager.getConnection(URL);
            System.out.println(" SUCCESS - Oracle Connected!");
            System.out.println("Database: " + con.getCatalog());
            return con;
        } catch (SQLException e) {
            System.err.println("FAILED: " + e.getMessage());
            return null;
        }
    }

    public static void main(String[] args) {
        System.out.println("1. STARTING ORACLE TEST");
        Connection con = getConnection();
        if (con != null) {
            System.out.println("2. ORACLE READY FOR CRUD!");
        }
        System.out.println("3. TEST COMPLETE");
    }
}
```

StudentCRUD.java

```
package mypack;
import java.sql.*;
import java.util.Scanner;
public class StudentCRUD {
    private static final String URL =
"jdbc:oracle:thin:system/vnrvjiet@localhost:1521/XEPDB1";
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

createTable();
int choice;
do {
    System.out.println("\n=== STUDENT CRUD ===");
    System.out.println("1. CREATE 2. READ 3. UPDATE 4. DELETE 5. Exit");
    System.out.print("Choose: ");
    choice = sc.nextInt();
    sc.nextLine(); // Clear buffer

    switch (choice) {
        case 1 -> createStudent(sc);
        case 2 -> readStudents();
        case 3 -> updateStudent(sc);
        case 4 -> deleteStudent(sc);
        case 5 -> System.out.println("Bye!");
        default -> System.out.println("Invalid!");
    }
} while (choice != 5);
sc.close();
}

public static void createTable() {
    try (Connection con = DriverManager.getConnection(URL)) {
        String sql = ""
            CREATE TABLE students (
                id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
                name VARCHAR2(100),
                age NUMBER,
                course VARCHAR2(100)
            )
            """;
        con.createStatement().execute(sql);
        System.out.println("Table ready!");
    } catch (SQLException e) {
        System.out.println("Table exists OK");
    }
}

public static void createStudent(Scanner sc) {
    try (Connection con = DriverManager.getConnection(URL)) {
        System.out.print("Name: ");
        String name = sc.nextLine();
        System.out.print("Age: ");
        int age = Integer.parseInt(sc.nextLine());
        System.out.print("Course: ");
        String course = sc.nextLine();
    }
}

```

```

        String sql = "INSERT INTO students (name, age, course) VALUES (?, ?, ?)";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, name); ps.setInt(2, age); ps.setString(3, course);
        ps.executeUpdate();
        System.out.println("Added!");
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void readStudents() {
    try (Connection con = DriverManager.getConnection(URL)) {
        ResultSet rs = con.createStatement().executeQuery("SELECT * FROM students");
        System.out.printf("%-5s %-15s %-5s %-15s\n", "ID", "NAME", "AGE",
"COURSE");
        System.out.println("-----");
        while (rs.next()) {
            System.out.printf("%-5d %-15s %-5d %-15s\n",
                rs.getInt("id"), rs.getString("name"), rs.getInt("age"), rs.getString("course"));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public static void updateStudent(Scanner sc) {
    try (Connection con = DriverManager.getConnection(URL)) {
        System.out.print("ID: ");
        int id = Integer.parseInt(sc.nextLine());
        System.out.print("New name: ");
        String name = sc.nextLine();
        PreparedStatement ps = con.prepareStatement("UPDATE students SET name = ?
WHERE id = ?");
        ps.setString(1, name); ps.setInt(2, id);
        if (ps.executeUpdate() > 0) System.out.println("Updated!");
        else System.out.println(" Not found");
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void deleteStudent(Scanner sc) {
    try (Connection con = DriverManager.getConnection(URL)) {
        System.out.print("ID: ");
        int id = Integer.parseInt(sc.nextLine());

```

```

        PreparedStatement ps = con.prepareStatement("DELETE FROM students WHERE id
= ?");
        ps.setInt(1, id);
        if (ps.executeUpdate() > 0) System.out.println("Deleted!");
        else System.out.println(" Not found");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
}

```

Output:

1. Output of DBConnection.java

1. STARTING ORACLE TEST

Connecting to Oracle 21c XE...

SUCCESS - Oracle Connected!

Database: null

2. ORACLE READY FOR CRUD!

3. TEST COMPLETE

Output of StudentCRUD.java

First Run (Table Creation)

Table ready!

=== STUDENT CRUD ===

1. CREATE 2. READ 3. UPDATE 4. DELETE 5. Exit

Choose:

3. CREATE Operation (Insert Record)

Choose: 1

Name: Rajesh

Age: 21

Course: CSE

Added!

4. READ Operation (Display Records)

Choose: 2

ID	NAME	AGE	COURSE
1	Rajesh	21	CSE

5. UPDATE Operation

Choose: 3

ID: 1

New name: Ramesh

Updated!

After Update (Read Again)

Choose: 2

ID	NAME	AGE	COURSE
----	------	-----	--------

1	Ramesh	21	CSE
---	--------	----	-----

6. DELETE Operation

Choose: 4

ID: 1

Deleted!

After Delete (Read Again)

Choose: 2

ID	NAME	AGE	COURSE
----	------	-----	--------

7. Exit Program

Choose: 5

Bye!

6. Create an xml for the bookstore. Validate the same using both DTD and XSD.

Program:

bookstore.dtd

```
<!ELEMENT bookstore (book+)>
<!-- Allow namespace attributes so DTD doesn't complain -->
<!ATTLIST bookstore
  xmlns:xsi CDATA #IMPLIED
  xsi:noNamespaceSchemaLocation CDATA #IMPLIED>
<!ELEMENT book (title, author, year, price)>
<!ATTLIST book category CDATA #REQUIRED>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

bookstore.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookstore SYSTEM "bookstore.dtd">
<bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="bookstore.xsd">
  <book category="programming">
    <title>Java Fundamentals</title>
    <author>James Gosling</author>
    <year>2022</year>
    <price>599.50</price>
  </book>
  <book category="database">
    <title>Mastering MySQL</title>
    <author>Michael Widenius</author>
    <year>2021</year>
    <price>450.00</price>
  </book>
</bookstore>
```

bookstore.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookstore SYSTEM "bookstore.dtd">
<bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="bookstore.xsd">
  <book category="programming">
    <title>Java Fundamentals</title>
    <author>James Gosling</author>
    <year>2022</year>
```

```
<price>599.50</price>
</book>
<book category="database">
  <title>Mastering MySQL</title>
  <author>Michael Widenius</author>
  <year>2021</year>
  <price>450.00</price>
</book>
</bookstore>
```

Output:

DTD Validation Output

DTD Validation Started...

No errors found.

XML is VALID according to DTD.

XSD Validation Output

XSD Validation Started...

Error: The document is not a valid XML Schema.

cvc-elt.1.a: Cannot find the declaration of element 'bookstore'.

Validation FAILED.

7.Design a Servlet-based controller that Connects the web application (Exercise 1) , Interacts with the database (Exercise 5)

cart.jsp

```
<%@ page import="java.util.*, model.CartItem" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Your Cart</title>
<style>
body{
    margin:0;
    font-family:'Segoe UI',sans-serif;
    background:#f1f3f6;
}
.header{
    background:#131921;
    color:white;
    padding:18px 40px;
    display:flex;
    justify-content:space-between;
    align-items:center;
}
.header a{
    color:white;
    text-decoration:none;
    margin-left:25px;
    font-weight:500;
}
.cart-container{
    width:80%;
    margin:40px auto;
}
.cart-card{
    background:white;
    padding:25px;
    border-radius:10px;
    box-shadow:0 4px 12px rgba(0,0,0,0.08);
}
table{
    width:100%;
    border-collapse:collapse;
}
th{
```

```
    text-align:left;
    padding:12px;
    border-bottom:2px solid #ddd;
}
td{
    padding:14px 10px;
    border-bottom:1px solid #eee;
}

.price{
    color:#2ecc71;
    font-weight:bold;
}
.btn{
    padding:8px 14px;
    border:none;
    border-radius:6px;
    cursor:pointer;
    font-weight:600;
}
.remove{
    background:#e74c3c;
    color:white;
}
.shop{
    background:#3498db;
    color:white;
}
.checkout{
    background:#f39c12;
    color:white;
    width:100%;
    margin-top:20px;
    padding:12px;
    font-size:16px;
}
.total-box{
    text-align:right;
    margin-top:20px;
    font-size:20px;
}
.empty{
    text-align:center;
    margin-top:100px;
```

```

}
    .empty button{
        padding:12px 20px;
        font-size:16px;
    }
</style>
</head>
<body>
<div class="header">
    <h2>Shopping Cart</h2>

    <div>
        <a href="catalog.jsp">Products</a>
        <a href="<%=request.getContextPath()%>/logout">Logout</a>
    </div>
</div>
<%
List<CartItem> cart =
(List<CartItem>)session.getAttribute("cart");
if(cart != null && !cart.isEmpty()){
double total = 0;
%>
<div class="cart-container">
<div class="cart-card">
<table>
<tr>
<th>Product</th>
<th>Price</th>
<th>Action</th>
</tr>
<%
for(CartItem item : cart){
total += item.getPrice();
%>
<tr>
<td>
<b><%=item.getName()%></b><br>
ID: <%=item.getId()%>
</td>
<td class="price">
<%=item.getPrice()%>
</td>
<td>
<form action="remove-from-cart" method="post">

```

```

<input type="hidden" name="id"
      value="<%=item.getId()%>">
<button class="btn remove">
Remove
</button>
</form>
</td>
</tr>
<%= } %>
</table>
<div class="total-box">
Total: <span class="price">
  <%=total%></span>
</div>
<button class="checkout">
Proceed to Checkout
</button>
</div>
</div>
<%=
} else {
%>
<div class="empty">
<h1>Your Cart is Empty </h1>
<p>Add some products to see them here.</p>
<a href="catalog.jsp">
<button class="shop">
Start Shopping
</button>
</a>
</div>
<%= } %>
</body>
</html>

```

catalog.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Product Catalog</title>
<style>
body{

```

```
margin:0;
font-family:Arial;
background:#f4f6f8;
}
.header{
display:flex;
justify-content:space-between;
align-items:center;
background:#222;
color:white;
padding:15px 30px;
}
.header a{
color:white;
text-decoration:none;
margin-left:20px;
}
.container{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(250px,1fr));
gap:25px;
padding:40px;
}
.card{
background:white;
border-radius:10px;
padding:20px;
text-align:center;
box-shadow:0 4px 10px rgba(0,0,0,0.1);
}
.card h3{
margin-bottom:10px;
}

.price{
color:green;
font-size:20px;
margin-bottom:15px;
}
button{
background:#ff9800;
border:none;
padding:10px 20px;
color:white;
```

```

border-radius:5px;
cursor:pointer;
}
button:hover{
background:#e68900;
}
</style>
</head>
<body>
<div class="header">
<h2>Shopping Cart</h2>
<div>
<a href="cart.jsp">View Cart</a>
<a href="<%=request.getContextPath()%>/logout">
Logout
</a>
</div>
</div>
<div class="container">
<div class="card">
<h3>Laptop</h3>
<p class="price">₹55,000</p>
<form action="<%=request.getContextPath()%>/cart"
method="post">
<input type="hidden" name="id" value="101">
<input type="hidden" name="name" value="Laptop">
<input type="hidden" name="price" value="55000">
<button type="submit">
Add to Cart
</button>
</form>
</div>
<div class="card">
<h3>Smartphone</h3>
<p class="price">₹25,000</p>
<form action="<%=request.getContextPath()%>/cart"
method="post">
<input type="hidden" name="id" value="102">
<input type="hidden" name="name" value="Smartphone">
<input type="hidden" name="price" value="25000">
<button type="submit">
Add to Cart
</button>
</form>

```

```

</div><div class="card">
  <h3>Headphones</h3>
  <p class="price">₹3,000</p>
  <form action="<%=request.getContextPath()%>/cart"
    method="post">
    <input type="hidden" name="id" value="103">
    <input type="hidden" name="name" value="Headphones">
    <input type="hidden" name="price" value="3000">
    <button type="submit">
      Add to Cart
    </button>
  </form>
</div>
<div class="card">
  <h3>Smart Watch</h3>
  <p class="price">₹5,000</p>
  <form action="<%=request.getContextPath()%>/cart"
    method="post">
    <input type="hidden" name="id" value="104">
    <input type="hidden" name="name" value="Smart Watch">
    <input type="hidden" name="price" value="5000">
    <button type="submit">
      Add to Cart
    </button>
  </form>
</div>
</div>
</body>
</html>

```

index.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Shopping Cart</title>

<style>

body{
  margin:0;

```

```
    font-family:'Segoe UI',sans-serif;
}

/* NAVBAR */
.navbar{
    background:#131921;
    color:white;
    padding:18px 40px;
    display:flex;
    justify-content:space-between;
    align-items:center;
}

.navbar a{
    color:white;
    text-decoration:none;
    margin-left:25px;
    font-weight:500;
}

/* HERO SECTION */
.hero{
    height:90vh;
    background:linear-gradient(to right,#4facfe,#00f2fe);
    display:flex;
    justify-content:center;
    align-items:center;
    text-align:center;
    color:white;
}

.hero h1 {
    font-size:52px;
    margin-bottom:10px;
}

.hero p{
    font-size:20px;
    margin-bottom:30px;
}

/* BUTTONS */
.btn{
    padding:14px 26px;
```

```

border:none;
border-radius:8px;
cursor:pointer;
font-size:16px;
margin:10px;
}

.login{
background:#f39c12;
color:white;
}

.register{
background:#2ecc71;
color:white;
}

.shop{
background:#ffffff;
color:#333;
font-weight:bold;
}
</style>
</head>
<body>
<!-- NAVBAR -->
<div class="navbar">
<h2> ShoppingCart</h2>
<div>
<a href="login.jsp">Login</a>
<a href="register.jsp">Register</a>
</div>
</div>
<!-- HERO -->
<div class="hero">
<div>
<h1>Welcome to ShoppingCart</h1>
<p>
Your one-stop destination for the best products at unbeatable prices.
</p>
<a href="products">
<button class="btn shop">
Start Shopping
</button>

```

```

</a>

<br>
<a href="login.jsp">
<button class="btn login">
Login
</button>
</a>
<a href="register.jsp">
<button class="btn register">
Register
</button>
</a>
</div>
</div>
</body>
</html>

```

login.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Login</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<div class="container">

<form class="form-box"
    action="<%=request.getContextPath()%>/login"
    method="post">

<h2>Login</h2>

<!-- ERROR BOX -->
<%
if(request.getParameter("error") != null){
%>
<div style="

```

```
background:#e74c3c;
color:white;
padding:12px;
margin-bottom:15px;
border-radius:6px;
text-align:center;
font-weight:600;">
Invalid Email or Password!
</div>
<% } %>
```

```
<!-- SUCCESS AFTER REGISTRATION -->
<%
if(request.getParameter("registered") != null){
%>
<div style="
background:#2ecc71;
color:white;
padding:12px;
margin-bottom:15px;
border-radius:6px;
text-align:center;
font-weight:600;">
Registration successful! Please login.
</div>
<% } %>
```

```
<!-- LOGOUT -->
<%
if(request.getParameter("logout") != null){
%>
<div style="
background:#3498db;
color:white;
padding:12px;
margin-bottom:15px;
border-radius:6px;
text-align:center;
font-weight:600;">
Logged out successfully!
</div>
<% } %>
<input type="email"
```

```

        name="email"
        placeholder="Enter Email"
        required>
<input type="password"
        name="password"
        placeholder="Enter Password"
        required>
<button type="submit">Login</button>
<p>
Don't have an account?
<a href="register.jsp">Register</a>
</p>
</form>
</div>
<!-- REAL POPUP ALERTS -->
<%
if(request.getParameter("error") != null){
%>
<script>
alert("Invalid Email or Password!");
</script>
<% } %>
<%
if(request.getParameter("registered") != null){
%>
<script>
alert("Registration Successful! Please login.");
</script>
<% } %>
<%
if(request.getParameter("logout") != null){
%>
<script>
alert("Logged out successfully!");
</script>
<% } %>
</body>
</html>

```

register.jsp

```

<%@ page contentType="text/html; charset=UTF-8" %>
<!DOCTYPE html>
<html>

```

```
<head>
<meta charset="UTF-8">
<title>Register</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">

<form class="form-box"
  action="<%=request.getContextPath()%>/register"
  method="post">
<h2>Create Account</h2>
<!-- ERROR POPUP -->
<%
if(request.getParameter("error") != null){
%>
<div style="
background:#e74c3c;
color:white;
padding:12px;
margin-bottom:15px;
border-radius:6px;
text-align:center;
font-weight:600;">
Registration Failed! Email may already exist.
</div>
<% } %>

<input type="text"
  name="name"
  placeholder="Username"
  required>
<input type="email"
  name="email"
  placeholder="Email"
  required>
<input type="password"
  name="password"
  placeholder="Password"
  required>
<button type="submit">Register</button>
<p>
Already have an account?
<a href="login.jsp">Login</a>
```

```
</p>
</form>
</div>
</body>
</html>
```

style.css

```
/* ----- Global Styles ----- */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background: #fff;
  color: #000;
}

h1, h2, h3 {
  margin: 0 0 10px 0;
}

a {
  color: #000;
  text-decoration: none;
}

/* ----- Forms (Login/Register) ----- */
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.form-box {
  background: #f2f2f2;
  padding: 20px;
  border-radius: 8px;
  width: 90%;
  max-width: 400px;
  display: flex;
  flex-direction: column;
}

.form-box input {
```

```
margin: 8px 0;
padding: 10px;
border-radius: 4px;
border: 1px solid #000;
}
```

```
.form-box button {
padding: 10px;
background: #000;
color: #fff;
border: none;
border-radius: 4px;
cursor: pointer;
margin-top: 10px;
}
```

```
.form-box p {
text-align: center;
margin-top: 10px;
}
```

```
/* ----- Catalog Page ----- */
.page-title {
text-align: center;
margin: 20px 0;
}
```

```
.products-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
gap: 15px;
padding: 0 15px 30px 15px;
}
```

```
.product-card {
background: #f2f2f2;
padding: 15px;
border-radius: 6px;
display: flex;
flex-direction: column;
}
```

```
.product-card h3 {
margin-bottom: 5px;
}
```

```
}
```

```
.product-card p {  
  margin-bottom: 8px;  
}
```

```
.product-card button {  
  padding: 8px;  
  background: #000;  
  color: #fff;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
  margin-top: auto;  
}
```

```
/* ----- Cart Page ----- */
```

```
.cart-container {  
  max-width: 600px;  
  margin: 20px auto;  
  padding: 0 15px;  
}
```

```
.cart-item {  
  display: flex;  
  justify-content: space-between;  
  background: #f2f2f2;  
  padding: 10px 15px;  
  margin-bottom: 10px;  
  border-radius: 6px;  
}
```

```
.cart-link {  
  display: inline-block;  
  margin: 20px auto;  
  padding: 10px 15px;  
  background: #000;  
  color: #fff;  
  text-decoration: none;  
  border-radius: 4px;  
  text-align: center;  
}
```

```
/* ----- Responsive ----- */
```

```

@media (max-width: 500px) {
  .products-grid {
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  }

  .cart-item {
    flex-direction: column;
    gap: 5px;
    align-items: flex-start;
  }
}

```

Output:

Create Account

Register

Already have an account? [Login](#)

Login

Login

Don't have an account? [Register](#)

Shopping Cart

View Cart Logout

Laptop

₹55,000

Add to Cart

Smartphone

₹25,000

Add to Cart

Headphones

₹3,000

Add to Cart

Smart Watch

₹5,000

Add to Cart

Product	Price	Action
Laptop ID: 101	55000.0	<button>Remove</button>
Smartphone ID: 102	25000.0	<button>Remove</button>

Total: 80000.0

[Proceed to Checkout](#)

8. Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)

Program:

index.html (Login Page)

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>
<h2>Login Page</h2>
<form action="login" method="post">
  Username: <input type="text" name="username" required><br><br>
  <input type="submit" value="Login">
</form>
</body>
</html>
```

LoginServlet.java (Creates Session + Cookie)

```
package com.example;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
  protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String user = request.getParameter("username");
    // Create HTTP Session
    HttpSession session = request.getSession();
    session.setAttribute("user", user);
    // Create Cookie
    Cookie cookie = new Cookie("username", user);
    cookie.setMaxAge(60 * 60); // 1 hour
    response.addCookie(cookie);
    response.sendRedirect("dashboard");
  }
}
```

DashboardServlet.java (Read Session + Cookie)

```
package com.example;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DashboardServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        // Get Session
        HttpSession session = request.getSession(false);
        String user = (String) session.getAttribute("user");
        // Get Cookies
        Cookie[] cookies = request.getCookies();
        String cookieUser = "";
        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("username")) {
                    cookieUser = c.getValue();
                }
            }
        }
        out.println("<h2>Welcome " + user + "</h2>");
        out.println("<h3>Session User: " + user + "</h3>");
        out.println("<h3>Cookie User: " + cookieUser + "</h3>");
        out.println("<a href='logout'>Logout</a>");
    }
}
```

LogoutServlet.java (Invalidate Session)

```
package com.example;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();
        }
    }
}
```

```

    }
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<h2>You are logged out</h2>");
    out.println("<a href='index.html'>Login Again</a>");
  }
}

```

web.xml

```

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
  <servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.example.LoginServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/login</url-pattern>
  </servlet-mapping>
  <servlet>
    <servlet-name>DashboardServlet</servlet-name>
    <servlet-class>com.example.DashboardServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>DashboardServlet</servlet-name>
    <url-pattern>/dashboard</url-pattern>
  </servlet-mapping>
  <servlet>
    <servlet-name>LogoutServlet</servlet-name>
    <servlet-class>com.example.LogoutServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>LogoutServlet</servlet-name>
    <url-pattern>/logout</url-pattern>
  </servlet-mapping>
</web-app>

```

Output:



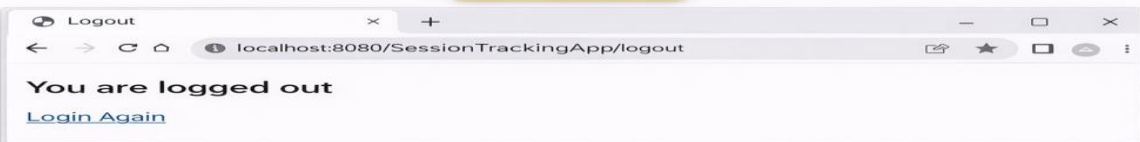
DASHBOARD



DASHBOARD



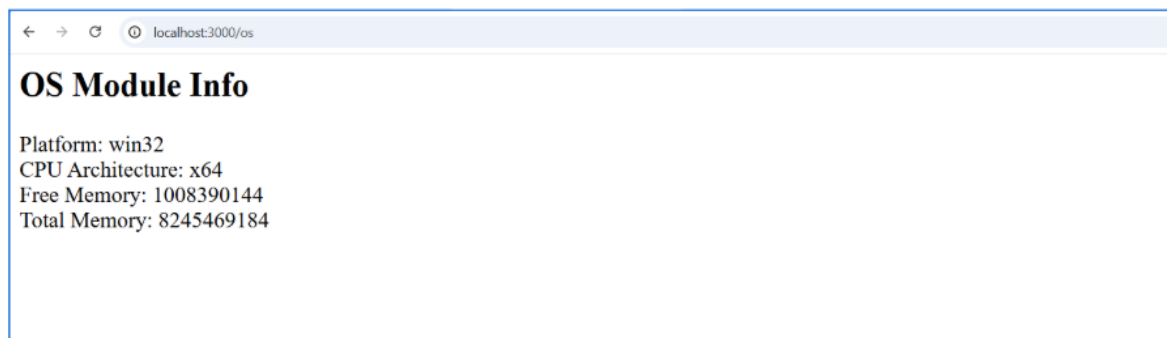
LOGOUT



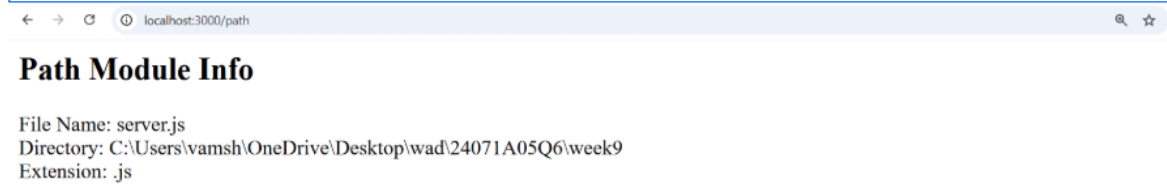
9. Create a custom server using http module and explore the other modules of Node JS like OS, path, event.

```
const http = require("http");
const os = require("os");
const path = require("path");
const EventEmitter = require("events");
const eventEmitter = new EventEmitter();
eventEmitter.on("request_received", (url) => {
  console.log(`Event Triggered: Request received for ${url}`);
});
const server = http.createServer((req, res) => {
  eventEmitter.emit("request_received", req.url);
  res.writeHead(200, { "Content-Type": "application/json" });
  if (req.url === "/") {
    res.end(JSON.stringify({ message: "Welcome to Custom Node Server" }));
  } else if (req.url === "/os") {
    res.end(JSON.stringify({
      platform: os.platform(),
      cpu_arch: os.arch(),
      cpus: os.cpus().length,
      free_memory: os.freemem(),
      total_memory: os.totalmem(),
      uptime: os.uptime()
    }));
  } else if (req.url === "/path") {
    const filePath = path.join(__dirname, "files", "demo.txt");
    res.end(JSON.stringify({
      joined_path: filePath,
      dirname: __dirname,
      ext: path.extname(filePath),
      basename: path.basename(filePath)
    }));
  } else if (req.url === "/event") {
    eventEmitter.emit("custom_event", "Hello Event!");
    eventEmitter.once("custom_event", (msg) => {
      console.log("Custom Event Fired:", msg);
    });
    res.end(JSON.stringify({ message: "Custom event triggered" }));
  } else {
    res.writeHead(404);
    res.end(JSON.stringify({ error: "Route not found" }));
  }
});
const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

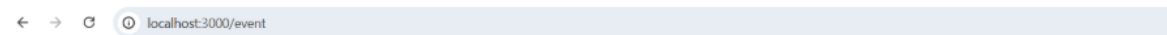
Output:



A screenshot of a web browser window. The address bar shows 'localhost:3000/os'. The page has a title 'OS Module Info'. Below the title, the following information is displayed: Platform: win32, CPU Architecture: x64, Free Memory: 1008390144, and Total Memory: 8245469184.



A screenshot of a web browser window. The address bar shows 'localhost:3000/path'. The page has a title 'Path Module Info'. Below the title, the following information is displayed: File Name: server.js, Directory: C:\Users\vamsh\OneDrive\Desktop\wad\24071A05Q6\week9, and Extension: .js.



A screenshot of a web browser window. The address bar shows 'localhost:3000/event'.

Event Module Demo

Check console for event message!

```
PS C:\Users\vamsh\OneDrive\Desktop\wad\24071A05Q6\week9> node server.js
Server running at http://localhost:3000
Request received for: /os
Request received for: /path
Request received for: /event
Event page visited at 5/4/2026, 9:14:43 pm
```

10. Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)

Program:

```
const express = require("express");
const app = express();
app.use(express.json());
const BASE_URL = "https://jsonplaceholder.typicode.com/users";
app.post("/students", async (req, res) => {
  try {
    const response = await fetch(BASE_URL, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(req.body)
    });
    const data = await response.json();
    res.json({
      message: "Student created",
      data
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.get("/students", async (req, res) => {
  try {
    const response = await fetch(BASE_URL);
    const data = await response.json();
    res.json(data);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.get("/students/:id", async (req, res) => {
  try {
    const response = await fetch(`${BASE_URL}/${req.params.id}`);
    const data = await response.json();
    res.json(data);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```

app.put("/students/:id", async (req, res) => {
  try {
    const response = await fetch(`${BASE_URL}/${req.params.id}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(req.body)
    });
    const data = await response.json();
    res.json({
      message: "Student updated",
      data
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.delete("/students/:id", async (req, res) => {
  try {
    await fetch(`${BASE_URL}/${req.params.id}`, {
      method: "DELETE"
    });
    res.json({
      message: "Student deleted"
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Output:

Server running on 
http://localhost:3000

CREATE (POST)

POST  http://localhost:3000/students  Save Send 

Params Authorization Headers (5) Body Pre-reqs Script Tests  Cookies Cat.  (ops 

JSON  JSON  Result

Body • Cookies (1) Headers (6) Test Results  Status: 201 Created Time: 183 ms: 105.8 

Pretty Raw Preview Visualize JSON  

```
1 {  
2   "message": "Student created",  
3   "data": {  
4     "name": "Rajesh",  
5     "email": "rajesh@gmail.com"  
6   }  
7 }
```



READ ALL

GET  http://localhost:3000/students  Save Send 

Params Authorization Headers (8) Body Pre-reqs Script Tests  Cookies Cat.  (ops 

JSON  JSON Whm  Result

Body • Cookies (5) Headers (6) Test Results  Status: 200 OK Time: 42 ms Size: 5351 B Save 

Pretty Raw Preview Visualize JSON  

```
1 {  
2   "id": 1,  
3   "name": "Leann Graham",  
4   "email": "Sincere@april.biz"  
5 },  
6   "id": 2,  
7   "name": "Ervin Howell",  
8   "email": "Shanna@melissa.tv"  
9 },  
10  "id": 11,  
11  "name": "Rajesh",  
12  "email": "rajesh@gmail.com"  
13 }
```



READ ONE

PUT  http://localhost:3000/students/11  Save Send 

Params Authorization Headers Body Pre-reqs Script Tests  Cookies Cat.  (ops 

JSON  JSON Whm  Result

UPDATE

11. For the above application create authorized end points using JWT (JSON Web Token).

Program:

```
const express = require("express");
const jwt = require("jsonwebtoken");
const app = express();
app.use(express.json());
const BASE_URL = "https://jsonplaceholder.typicode.com/users";
const SECRET = "mysecretkey";
const USER = {
  username: "admin",
  password: "1234"
};
function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  const token = authHeader && authHeader.split(" ")[1];
  if (!token) return res.status(401).json({ error: "Token required" });
  jwt.verify(token, SECRET, (err, user) => {
    if (err) return res.status(403).json({ error: "Invalid token" });
    req.user = user;
    next();
  });
}
app.post("/login", (req, res) => {
  const { username, password } = req.body;
  if (username !== USER.username || password !== USER.password) {
    return res.status(401).json({ error: "Invalid credentials" });
  }
  const token = jwt.sign({ username }, SECRET, { expiresIn: "1h" });
  res.json({ message: "Login successful", token });
});
app.post("/students", authenticateToken, async (req, res) => {
  try {
    const response = await fetch(BASE_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(req.body)
    });
    const data = await response.json();
    res.json({ message: "Created", data });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```

});
app.get("/students", authenticateToken, async (req, res) => {
  try {
    const response = await fetch(BASE_URL);
    const data = await response.json();
    res.json(data);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.get("/students/:id", authenticateToken, async (req, res) => {
  try {
    const response = await fetch(`${BASE_URL}/${req.params.id}`);
    const data = await response.json();
    res.json(data);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.put("/students/:id", authenticateToken, async (req, res) => {
  try {
    const response = await fetch(`${BASE_URL}/${req.params.id}`, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(req.body)
    });
    const data = await response.json();
    res.json({ message: "Updated", data });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.delete("/students/:id", authenticateToken, async (req, res) => {
  try {
    await fetch(`${BASE_URL}/${req.params.id}`, {
      method: "DELETE"
    });
    res.json({ message: "Deleted" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

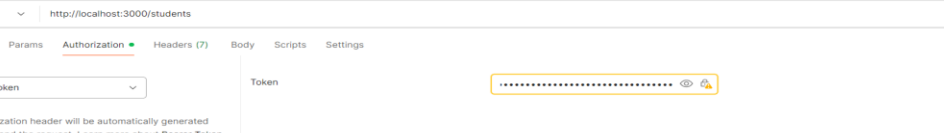
```

Output:

[illegible]

The screenshot displays the Swagger UI for a REST API. The top navigation bar includes links for Overview, POST http://localhost:3000/students, GET http://localhost:3000/students, and a plus icon for additional endpoints. The left sidebar contains a 'Collections' section and an 'Environments' section with a 'Mock servers' tab. The main area shows a GET request to http://localhost:3000/students. The 'Auth Type' is set to 'No Auth'. The response body is displayed in JSON format, showing a 401 Unauthorized status with the message 'error: Access denied. No token provided.' The response headers include 'Content-Type: application/json' and 'Status: 401 Unauthorized'.

The screenshot displays the Postman application interface. On the left is a vertical sidebar with navigation icons and labels: Overview, Collections, Environments, History, Mock Servers, Specs, Monitors, Plans, Insights, Files, and Help. The main workspace is divided into several sections. At the top, a status bar shows the request method (GET) and the URL (http://localhost:3000/students). Below this, there are tabs for Docs, Params, Authorization, Headers (6), Body, Scripts, and Settings. The Authorization tab is currently selected, showing 'Auth Type' set to 'No Auth'. The right side of the workspace displays the response details for the GET request. It shows a status of '401 Unauthorized' with a response time of 0 ms and a size of 250 B. The response body is a JSON object: {"error": "Access denied. No token provided."}. The bottom of the workspace has tabs for Body, Cookies, Headers (7), Test Results, and a toggle for 'Pass the correct auth credentials'.



The screenshot shows the Postman interface with a GET request to `http://localhost:3000/students`. The request is successful, returning a 200 OK status with a response time of 24 ms and a body size of 334 B. The response body is a JSON array of two student objects. The 'Authorization' tab is selected in the request header section, and the 'Bearer Token' is visible. The 'Body' tab is selected in the response section, showing the JSON data.

```

{
  "id": 1,
  "name": "John Doe",
  "age": 20,
  "grade": "A"
},
{
  "id": 2,
  "name": "Jane Smith",
  "age": 22,
  "grade": "B"
}

```

12. Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.

```
import React from "react";
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";
import "bootstrap/dist/css/bootstrap.min.css";
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark">
      <div className="container">
        <Link className="navbar-brand" to="/">SMS</Link>
        <div className="collapse navbar-collapse">
          <ul className="navbar-nav ms-auto">
            <li className="nav-item"><Link className="nav-link" to="/">Home</Link></li>
            <li className="nav-item"><Link className="nav-link"
to="/register">Register</Link></li>
            <li className="nav-item"><Link className="nav-link"
to="/login">Login</Link></li>
            <li className="nav-item"><Link className="nav-link"
to="/contact">Contact</Link></li>
            <li className="nav-item"><Link className="nav-link"
to="/about">About</Link></li>
          </ul>
        </div>
      </div>
    </nav>
  );
}
function Home() {
  return (
    <div className="container mt-5 text-center">
      <h1 className="mb-3">Student Management System</h1>
      <p className="lead">Manage students efficiently with a simple UI</p>
    </div>
  );
}
function Register() {
  return (
    <div className="container mt-5">
      <div className="card shadow p-4 mx-auto" style={{ maxWidth: "400px" }}>
        <h3 className="mb-3 text-center">Register</h3>
        <form>
          <div className="mb-3">
```

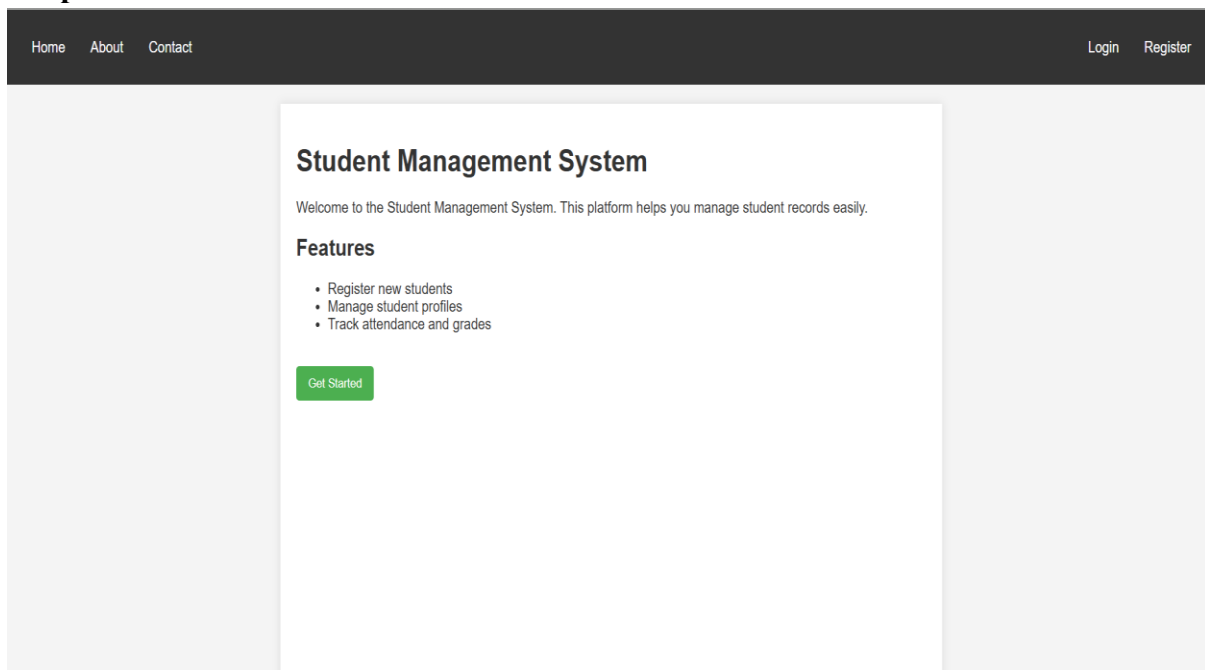


```

return (
  <div className="container mt-5">
    <div className="card shadow p-4">
      <h3>About</h3>
      <p>This is a simple Student Management System UI built using React and
Bootstrap.</p>
    </div>
  </div>
);
}
export default function App() {
  return (
    <Router>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/register" element={<Register />} />
        <Route path="/login" element={<Login />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
}

```

Output:



[Home](#)[About](#)[Contact](#)[Login](#)[Register](#)

Contact Us

Name:

Email:

Message:

[Submit](#)

Our Details
Email: admin@school.com
Phone: 123-456-7890

[Home](#)[About](#)[Contact](#)[Login](#)[Register](#)

Login

Email ID:

Password:

[Login](#)

[Home](#)[About](#)[Contact](#)[Login](#)[Register](#)

Register

First Name:

Last Name:

Email:

Password:

[Register](#)

13. Create a service in react that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js

Program:

WeatherChat.jsx

```
import {
  Chart as ChartJS,
  LineElement,
  CategoryScale,
  LinearScale,
  PointElement,
} from "chart.js";
import { Line } from "react-chartjs-2";

ChartJS.register(LineElement, CategoryScale, LinearScale, PointElement);

function WeatherChart({ data }) {
  if (!data) return null;

  const labels = data.list.map(item =>
    new Date(item.dt * 1000).toLocaleDateString()
  );

  const temps = data.list.map(item => item.main.temp);

  const chartData = {
    labels,
    datasets: [
      {
        label: "Temperature (°C)",
        data: temps,
        borderColor: "blue",
        fill: false,
      },
    ],
  };

  return <Line data={chartData} />;
}

export default WeatherChart;
```

WeatherApp.jsx

```
import { useState } from "react";
import { getCurrentWeather, getHistoricalWeather } from "../services/weatherService";
import WeatherChart from "../components/WeatherChart";

function WeatherApp() {
```

```

const [city, setCity] = useState("");
const [current, setCurrent] = useState(null);
const [history, setHistory] = useState(null);

const fetchWeather = async () => {
  const currentData = await getCurrentWeather(city);
  setCurrent(currentData);

  const { coord } = currentData;
  const historyData = await getHistoricalWeather(coord.lat, coord.lon);
  setHistory(historyData);
};

return (
  <div style={{ textAlign: "center" }}>
    <h2>Weather Dashboard</h2>

    <input
      type="text"
      placeholder="Enter city"
      value={city}
      onChange={(e) => setCity(e.target.value)}
    />
    <button onClick={fetchWeather}>Search</button>

    {current && (
      <div>
        <h3>{current.name}</h3>
        <p>Temperature: {current.main.temp} °C</p>
        <p>Condition: {current.weather[0].description}</p>
      </div>
    )}
    <WeatherChart data={history} />
  </div>
);
}
export default WeatherApp;

```

Output:



14. Create a TODO application in react with necessary components and deploy it into github.

Program:

TodoInput.jsx

```
import { useState } from "react";
function TodoInput({ addTodo }) {
  const [text, setText] = useState("");
  const [dueDate, setDueDate] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    if (!text.trim()) return;
    addTodo({
      id: Date.now(),
      text,
      completed: false,
      dueDate,
    });
    setText("");
    setDueDate("");
  };
  return (
    <form onSubmit={handleSubmit} className="mb-3">
      <div className="row g-2">
        <div className="col-12">
          <input
            className="form-control"
            type="text"
            placeholder="Enter task"
            value={text}
            onChange={(e) => setText(e.target.value)}
          />
        </div>
        <div className="col-8">
          <input
            className="form-control"
            type="date"
            value={dueDate}
            onChange={(e) => setDueDate(e.target.value)}
          />
        </div>
        <div className="col-4 d-grid">
          <button className="btn btn-primary" type="submit">
            Add
          </button>
        </div>
      </div>
    </form>
  );
}
```

```

        </button>
      </div>
    </div>
  </form>
);}

```

export default TodoInput;

TodoItem.jsx

```
import { useState } from "react";
```

```

function TodoItem({ todo, toggleTodo, deleteTodo, updateTodo }) {
  const [isEditing, setIsEditing] = useState(false);
  const [newText, setNewText] = useState(todo.text);
  const [newDate, setNewDate] = useState(todo.dueDate || "");
  const today = new Date().toISOString().split("T")[0];
  const isOverdue =
    !todo.completed &&
    todo.dueDate &&
    todo.dueDate < today;
  const handleUpdate = () => {
    updateTodo(todo.id, newText, newDate);
    setIsEditing(false);
  };
  return (
    <div
      className={`d-flex justify-content-between align-items-center p-2 mb-2 rounded ${
        isOverdue ? "bg-danger-subtle" : "bg-light"
      }`}
    >
      {isEditing ? (
        <div className="d-flex gap-2 w-100">
          <input
            className="form-control"
            value={newText}
            onChange={(e) => setNewText(e.target.value)}
          />
          <input
            className="form-control"
            type="date"
            value={newDate}
            onChange={(e) => setNewDate(e.target.value)}
          />
          <button className="btn btn-success" onClick={handleUpdate}>
            Save

```

```

        </button>
      </div>
    ) : (
      <div>
        <span
          onClick={() => toggleTodo(todo.id)}
          style={{
            cursor: "pointer",
            textDecoration: todo.completed
              ? "line-through"
              : "none",
          }}
          className={isOverdue ? "text-danger" : ""}
        >
          {todo.text}
          {todo.dueDate && ` (Due: ${todo.dueDate})`}
        </span>

        <div className="btn-group">
          <button
            className="btn btn-sm btn-warning"
            onClick={() => setIsEditing(true)}
          >
            Edit
          </button>
          <button
            className="btn btn-sm btn-danger"
            onClick={() => deleteTodo(todo.id)}
          >
            Delete
          </button>
        </div>
      </div>
    )}
  </div>
);}

```

export default TodoItem;

TodoList.jsx

```

import TodoItem from "../TodoItem";
function TodoList({ todos, toggleTodo, deleteTodo, updateTodo }) {
  return (
    <div>
      {todos.map((todo) => (

```

```

    <TodoItem
      key={todo.id}
      todo={todo}
      toggleTodo={toggleTodo}
      deleteTodo={deleteTodo}
      updateTodo={updateTodo}
    />
  ))}
</div>

);}
export default TodoList;

```

App.jsx

```

import { useState } from "react";
import TodoInput from "../components/TodoInput";
import TodoList from "../components/TodoList";
function App() {
  const [todos, setTodos] = useState([]);
  const addTodo = (todo) => {
    setTodos([...todos, todo]);
  };
  const toggleTodo = (id) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id
          ? { ...todo, completed: !todo.completed }
          : todo
      )
    );
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter((todo) => todo.id !== id));
  };
  const updateTodo = (id, newText, newDate) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id
          ? { ...todo, text: newText, dueDate: newDate }
          : todo
      )
    );
  };
  return (
    <div className="container d-flex justify-content-center align-items-center min-vh-100">
      <div className="card shadow p-4" style={{ width: "500px" }}>
        <h2 className="text-center mb-4">Todo App</h2>

        <TodoInput addTodo={addTodo} />

```

```

<TodoList
  todos={todos}
  toggleTodo={toggleTodo}
  deleteTodo={deleteTodo}
  updateTodo={updateTodo}
/>
</div>
</div>
);}
export default App;

```

Output:

